

Módulo 5 – Capítulo 05 – Sección 01

El desafío de testear sistemas no deterministas: estrategias y restricciones

El testing de software convencional descansa sobre un supuesto que los sistemas de IA violan por diseño: que la misma entrada produce siempre la misma salida. `assertEqual(add(2, 3), 5)` es un test correcto porque `add` es determinista. `assertEqual(llm.generate("Resume este documento"), expected_summary)` es un test frágil porque el LLM con temperatura mayor a cero producirá variantes diferentes en cada ejecución, todas potencialmente correctas. Esta incompatibilidad fundamental entre el modelo de testing tradicional y la naturaleza estocástica de los LLMs no significa que no se pueda testear sistemas de IA: significa que el paradigma de testing debe adaptarse para evaluar propiedades y criterios de calidad en lugar de igualdad exacta de strings.

El no-determinismo en sistemas de IA se manifiesta en tres capas distintas que requieren estrategias diferentes. La variabilidad estocástica del modelo es la más obvia: con temperatura mayor a cero, el mismo prompt produce respuestas diferentes en cada ejecución. Con temperatura 0 y seed fijo en OpenAI, el modelo es casi determinista mientras la infraestructura del proveedor no cambie, lo que permite tests de snapshot sobre un subconjunto de casos críticos. La variabilidad de versiones del modelo es la más insidiosa: cuando el proveedor actualiza silenciosamente el modelo manteniendo el mismo identificador —o cuando se pina el modelo con fecha exacta y esa fecha expiró—, el comportamiento puede cambiar sin ningún cambio en el código de la aplicación. Monitorear el campo `model` en cada respuesta y alertar cuando difiere del esperado detecta estas actualizaciones silenciosas. La variabilidad del prompt es la más controlable: cambios menores en el texto del prompt pueden cambiar significativamente el comportamiento, lo que refuerza la importancia del versionado de prompts discutido en el capítulo 6.

La estrategia de testing para sistemas no deterministas se articula en cuatro enfoques complementarios. El testing por propiedades verifica condiciones estructurales sobre la respuesta sin comparar el texto exacto: `assert len(response) > 50`, `assert json.loads(response)["category"] in VALID_CATEGORIES`, `assert "precio" in`

`response.lower()` . Las propiedades estructurales son más robustas que la igualdad exacta porque permiten variación léxica legítima mientras detectan fallos reales. El testing basado en LLM-as-judge usa otro modelo para evaluar si la respuesta cumple los criterios de calidad: "¿Esta respuesta responde correctamente la pregunta dada este contexto? Responde solo SÍ o NO." El umbral estadístico trata el test como un experimento probabilístico: ejecutar el mismo caso de prueba 10 a 20 veces y verificar que el criterio de calidad se cumple al menos el 90% de las veces. El snapshot testing con temperatura 0 captura la respuesta de referencia con parámetros deterministas y la usa como señal de cambio de comportamiento.

Aspectos técnicos del testing no determinista

- **Testing por propiedades:** en lugar de `assert response == expected` , verificar `assert len(response) > 50` , `assert "precio" in response.lower()` , `assert json.loads(response) ["category"] in VALID_CATEGORIES` ; más robusto ante variación léxica legítima.
- **Seeds para reproducibilidad parcial:** `seed=42` en OpenAI hace el sampling determinista en ausencia de actualizaciones de infraestructura; útil para snapshots temporales de comportamiento, frágil ante actualizaciones del modelo.
- **Umbral estadístico de pass/fail:** ejecutar el mismo caso N=10-20 veces y verificar que pasa al menos el X% de las veces (ej. 90%); apropiado para funcionalidades críticas donde la consistencia estadística importa más que el 100% de precisión.
- **Mocks del LLM para unit tests:** respuestas ficticias predefinidas en lugar del LLM real para testear la lógica de parsing, validación y manejo de errores sin incurrir en latencia ni costo de API; los mocks son la base de la pirámide de testing de IA.
- **Evaluación con criterios explícitos:** definir criterios de calidad textuales ("¿la respuesta menciona al menos tres beneficios del producto?", "¿la respuesta está en el idioma del usuario?") verificables con regex, parsers o un LLM-as-judge configurado.

El objetivo del testing en sistemas de IA no es verificar que la respuesta es exactamente la esperada, sino verificar que satisface los criterios de calidad del negocio con suficiente consistencia estadística bajo condiciones controladas. Esta perspectiva hace al testing de IA más cercano al testing estadístico de sistemas de aprendizaje automático que al testing unitario de software determinista, y requiere adoptar esa mentalidad desde el inicio del proyecto.

Principio rector: El objetivo del testing en sistemas de IA no es la igualdad exacta sino la verificación de propiedades de calidad con suficiente consistencia estadística; la adopción de esta perspectiva transforma el testing de una actividad frustrante —el test siempre falla por variabilidad— en una actividad rigurosa que da confianza real en el comportamiento del sistema.

Módulo 5 – Capítulo 05 – Sección 02

Unit testing de componentes de IA: mocks, fixtures y boundaries

La estrategia de unit testing para sistemas de IA es conceptualmente simple una vez que se identifica el principio correcto: testear el código propio, no el modelo. El comportamiento del LLM es una caja negra externa fuera del control del test; lo que el equipo puede controlar —y debe testear exhaustivamente— es todo el código Python que rodea a esa caja negra: la construcción del prompt con las variables correctamente interpoladas, el parseo de la respuesta en la estructura de datos esperada, la validación de la salida contra el schema, el manejo de los errores que el LLM puede devolver, y la lógica de retry cuando el parseo falla. Todo este código es determinista, testeable con mocks y completamente bajo el control del equipo de desarrollo.

El boundary principal a identificar es la función o método que llama al SDK del proveedor: `client.messages.create()` en Anthropic o `client.chat.completions.create()` en OpenAI. Mockeando este método con `unittest.mock.MagicMock()` o `pytest-mock`, los tests ejecutan toda la lógica de Python sin hacer ninguna llamada a la red, sin incurrir en costo de API, sin depender de la disponibilidad del servicio externo, y con latencia de milisegundos en lugar de segundos. El mock se configura para devolver una respuesta predefinida que simula el formato real del SDK: un objeto `Message` de Anthropic o un objeto `ChatCompletion` de OpenAI con los campos que el código de producción espera leer.

Los fixtures en pytest son el mecanismo para organizar los datos de prueba: respuestas del LLM en diferentes formatos, casos de input normal y de input límite, payloads de error que el SDK puede devolver. Almacenar estos fixtures como archivos JSON en `tests/fixtures/` — `valid_json_response.json`, `invalid_json_response.json`, `rate_limit_error.json` — hace que los tests sean autodocumentados y que los casos de prueba sean reproducibles por cualquier miembro del equipo sin acceso a las credenciales de API. El decorador `@pytest.mark.parametrize("input_text, expected_category", [...])` ejecuta el mismo test con docenas de combinaciones de input/output esperado sin duplicar el código de test, que es

especialmente valioso para tests de clasificación o extracción donde los casos borde son numerosos.

La cobertura de tests en componentes de IA debe enfocarse en las ramas del código de Python, no en las variaciones del comportamiento del modelo. Cada rama del constructor de prompts —la que añade el contexto del usuario cuando existe, la que usa el prompt por defecto cuando no existe—, cada rama del parser —la que parsea JSON válido, la que maneja JSON inválido, la que maneja una respuesta vacía—, y cada rama del manejo de errores —la que reintenta ante un 429, la que propaga inmediatamente un 401— deben tener al menos un test. El 90% de cobertura de branches en el código Python que rodea al LLM es alcanzable y deseable; el 100% de cobertura del comportamiento del LLM es imposible y no es el objetivo.

Conceptos clave del unit testing de IA

- **Mocking del SDK con `pytest-mock`**: `mock.patch.object(anthropic_client, 'messages')` reemplaza el método `create` del cliente con un `MagicMock`; `mock.return_value = build_mock_message(text="Respuesta de prueba")` configura lo que devuelve; el test verifica que el código llama al mock con los parámetros correctos.
- **Fixtures de respuestas del LLM**: archivos JSON en `tests/fixtures/` con respuestas representativas —JSON bien formado, JSON inválido, respuesta truncada, respuesta con caracteres especiales—; cargados con `@pytest.fixture` y pasados como parámetros a los tests que los necesitan.
- **`@pytest.mark.parametrize`**: ejecuta el mismo test con múltiples combinaciones de input/output; ideal para clasificadores, extractores de entidades y cualquier componente donde la lógica es la misma pero los datos varían; evita la duplicación de código de test.
- **Testing del manejo de errores**: verificar que el código captura correctamente `anthropic.RateLimitError`, `anthropic.APITimeoutError`, `json.JSONDecodeError` y respuestas vacías; estos paths de error son los más críticos y los menos ejercitados en tests de integración.
- **Verificación de la construcción del prompt**: `assert "nombre_del_usuario" in calls[0][1]['messages'][1]['content']`; verificar que el prompt construido contiene las variables interpoladas, tiene la estructura de roles esperada y no excede el límite de tokens configurado.

Nota del Arquitecto: El valor más alto del unit testing en sistemas de IA no está en los casos de éxito —el código que funciona cuando el LLM devuelve lo esperado— sino

en los casos de error: el JSON inválido, el timeout, el 429 en producción, la respuesta vacía. Estos son los casos que el código de producción encontrará inevitablemente y que, si no están testeados, se convierten en bugs silenciosos o en excepciones sin manejar que llegan al usuario final. Un test de que el código maneja correctamente una respuesta JSON inválida es más valioso que diez tests de que procesa correctamente respuestas bien formadas.

La siguiente sección eleva el nivel de los tests hacia la integración completa de cadenas y flujos, donde los componentes individuales —correctamente testeados con mocks en los unit tests— se verifican funcionando en conjunto con LLMs reales sobre datasets curados.

Para recordar: El valor del unit testing en IA está en testear el código propio —constructores de prompts, parsers de respuestas, validadores de salida, lógica de retry— con mocks del LLM que simulan el rango completo de respuestas posibles, incluyendo los casos de error que el modelo real producirá inevitablemente en producción.

Módulo 5 – Capítulo 05 – Sección 03

Testing de integración: validación de cadenas y flujos completos

Los tests de integración en sistemas de IA son la capa donde se descubren los problemas que los unit tests con mocks no pueden ver: el retriever que devuelve documentos irrelevantes porque el índice vectorial tiene embeddings generados con el modelo equivocado, el prompt que pierde variables de interpolación cuando el contexto excede cierto tamaño, o el parser que funciona correctamente con los fixtures de test pero falla ante formatos de respuesta que el modelo real produce con frecuencia pero el mock nunca simuló. Estos bugs de integración son los más costosos de encontrar en producción porque requieren tráfico real para manifestarse; los tests de integración son el mecanismo que los detecta antes.

A diferencia de los unit tests que usan mocks del LLM, los tests de integración hacen llamadas reales a los proveedores —al LLM, al vector store, a la base de datos— para verificar que los componentes funcionan correctamente en conjunto. Esto introduce tres consideraciones operacionales. El costo real: cada test de integración consume tokens y genera una factura; usar modelos económicos como `claude-3-haiku-20240307` o `gpt-4o-mini` reservados para el entorno de test y configurar `TEST_MAX_TOKENS=256` en variables de entorno para limitar la longitud de las respuestas mantiene el costo por run del orden de centavos en lugar de dólares. La latencia real: los tests de integración toman segundos, no milisegundos; una suite de 50 casos puede tardar de 2 a 5 minutos, lo que los hace candidatos para ejecución selectiva en CI en lugar de ejecución en cada commit. La dependencia de la red: los tests de integración fallan ante degradaciones del proveedor, lo que requiere distinguir entre un fallo genuino del sistema bajo prueba y un fallo de la dependencia externa; reintentar automáticamente los tests fallidos una vez antes de reportarlos como error es una práctica razonable.

El dataset de tests de integración —conjunto de pares (input, criterio de calidad) representativos del dominio real— es el activo más importante de la suite y debe tratarse con el mismo cuidado que el código: versionado en el repositorio como `tests/eval/dataset.json`, con revisión en PRs cuando se añaden o modifican casos, y con crecimiento orgánico que incorpora los casos de fallo observados en producción. Un buen

dataset incluye casos normales (preguntas dentro del dominio con respuesta directa en el corpus), casos límite (preguntas que requieren inferencia sobre múltiples documentos), casos negativos (preguntas fuera del dominio donde el sistema debe reconocer que no tiene información), y casos de adversarial testing (formulaciones que históricamente causaron problemas).

Los tests de integración para un sistema RAG verifican la cadena completa: que la pregunta del dataset genera una query de embeddings que recupera al menos un fragmento relevante con score de similitud sobre el umbral configurado, que el prompt se construye con ese fragmento en la posición correcta dentro del contexto, y que la respuesta del LLM referencia información del documento recuperado en lugar de inventar datos. Esta cadena de verificación hace explícitas las dependencias entre componentes y detecta cuando cambiar un componente —por ejemplo, el modelo de embeddings de `text-embedding-3-small` a `text-embedding-3-large`— mejora la calidad del retriever pero introduce un comportamiento diferente en el synthesizer.

Aspectos técnicos del testing de integración

- **Dataset de evaluación curado:** 50 a 200 pares (input, criterio de calidad) en `tests/eval/dataset.json`; incluye casos normales, límite, negativos y adversariales; crece con los fallos de producción; se versiona en Git con el mismo rigor que el código.
- **Tests de cadena RAG:** verificar que el retriever devuelve fragmentos con score > threshold, que el prompt incluye esos fragmentos, y que la respuesta referencia información del contexto (faithfulness básica); estos tres checks detectan los fallos más comunes de los sistemas RAG.
- **Comparación A/B de versiones de prompt:** ejecutar el mismo dataset con dos versiones de prompt y comparar las métricas de calidad; el criterio de aprobación es que la nueva versión no degrada el score medio en más del 3% respecto al baseline.
- **Validación del schema de salida:** para endpoints que devuelven JSON, verificar que el 100% de las respuestas del dataset son JSON válido parseado correctamente por el schema Pydantic; una tasa de éxito de parseo inferior al 98% indica un problema de instrucción al modelo.
- **Activación condicional en CI:** configurar el pipeline de CI para ejecutar los tests de integración solo cuando los archivos en `prompts/`, `pipelines/`, `config/` o `tests/eval/` cambian, evitando el costo de ejecutarlos en cada commit de código de interfaz o documentación.

La frontera entre unit tests e integration tests no es una línea dura sino un espectro: algunos tests de integración son suficientemente rápidos y baratos para ejecutarse en cada PR, otros solo en releases. La clave es que la suite de integración crezca con el sistema y que cada caso de fallo importante en producción se convierta en un test de integración que detecte esa regresión en el futuro.

Principio rector: Los tests de integración son el único mecanismo que detecta los problemas en la interacción entre componentes reales: el retriever que devuelve documentos irrelevantes, el prompt que pierde variables al escalar, o el parser que falla ante formatos que el modelo real produce pero el mock no simula.

Módulo 5 – Capítulo 05 – Sección 04

Evaluación de calidad de respuestas: métricas automáticas y humanas

La calidad de una respuesta de IA no es una dimensión única: una respuesta puede ser fiel al contexto pero irrelevante para la pregunta, puede ser relevante pero factualmente incorrecta, puede ser correcta pero mal formateada para el caso de uso, puede satisfacer todos los criterios técnicos pero ser inapropiada en tono para la audiencia del sistema. Esta multidimensionalidad de la calidad hace que ninguna métrica única capture todo lo que importa, y que el sistema de evaluación de un sistema de IA requiera al menos tres métricas complementarias que cubran dimensiones diferentes.

Las métricas basadas en referencia —que comparan la respuesta generada con una respuesta de referencia conocida— son las más intuitivas pero las más costosas de construir: requieren un dataset de respuestas doradas anotadas por expertos del dominio. BLEU y ROUGE, desarrolladas para evaluación de traducción automática y resumen, miden la superposición de n-gramas entre la respuesta generada y la de referencia; son computacionalmente baratas pero penalizan la variación léxica legítima. BERTScore mejora sustancialmente sobre BLEU/ROUGE usando embeddings contextuales de BERT para medir similitud semántica: dos respuestas que expresan la misma idea con vocabulario diferente obtienen un BERTScore alto aunque su BLEU sea bajo.

Las métricas sin referencia eliminan el requisito de respuestas doradas y escalan mejor para datasets grandes. El patrón LLM-as-judge usa un modelo evaluador —frecuentemente un modelo más capaz que el evaluado, como GPT-4 evaluando un sistema basado en GPT-4o-mini— para puntuar la calidad de la respuesta según una rúbrica explícita. RAGS implementa un conjunto de métricas sin referencia específicamente diseñadas para RAG: `faithfulness` descompone la respuesta en afirmaciones atómicas y verifica qué porcentaje puede inferirse del contexto recuperado; `answer_relevancy` genera preguntas a partir de la respuesta y mide su similitud con la pregunta original, penalizando respuestas que responden algo diferente a lo preguntado; `context_recall` verifica qué fracción de la información necesaria para responder aparece en el contexto recuperado, detectando fallos del retriever.

La evaluación humana sigue siendo el gold standard que calibra y valida las métricas automáticas. Ninguna métrica automática captura con perfecta fidelidad el juicio de un experto del dominio, y la correlación entre métricas automáticas y juicio humano —medida con Spearman correlation o Cohen's Kappa— es el criterio de validez de las métricas automáticas para ese caso de uso específico. El proceso de evaluación humana tiene tres componentes críticos: la selección de evaluadores con conocimiento del dominio, el diseño de rúbricas explícitas con ejemplos ancla para cada nivel de la escala (no descriptores abstractos como "bueno" o "malo"), y la medición del inter-annotator agreement para verificar que la tarea de evaluación es suficientemente objetiva. Un inter-annotator agreement medido con Cohen's Kappa inferior a 0.6 indica que la rúbrica es ambigua y necesita refinamiento antes de que las anotaciones tengan valor.

Métricas automáticas y humanas de calidad

- **RAGAS `faithfulness`** : proporción de afirmaciones de la respuesta que pueden inferirse del contexto recuperado; detecta alucinaciones en sistemas RAG; score de 0 a 1, donde valores inferiores a 0.7 indican problemas graves de groundedness.
- **RAGAS `answer_relevancy`** : similitud coseno entre la pregunta original y las preguntas generadas a partir de la respuesta; penaliza respuestas que desvían el tema; valores inferiores a 0.7 indican que el sistema tiende a responder preguntas diferentes a las formuladas.
- **BERTScore F1**: combina precision y recall a nivel de embeddings contextuales; más robusto que BLEU ante variación léxica y más interpretable que ROUGE-L para dominios con vocabulario especializado.
- **LLM-as-judge con rúbrica explícita**: prompt de evaluación con escala de 1 a 5 con ejemplos concretos por nivel; evaluar criterios separados (completitud, precisión, formato) en lugar de holístico para reducir el halo effect; calibrar con muestras de evaluación humana.
- **Inter-annotator agreement**: Cohen's Kappa o Krippendorff's Alpha entre pares de evaluadores sobre la misma muestra; valores superiores a 0.6 indican acuerdo sustancial; valores entre 0.4-0.6 indican acuerdo moderado que requiere refinamiento de la rúbrica.

El sistema de evaluación de calidad no es un ejercicio académico sino el mecanismo operacional que informa las decisiones de ingeniería: cambiar el modelo, refinar el prompt, ajustar el retriever, o añadir ejemplos few-shot. La siguiente sección aborda el regression testing: cómo usar el sistema de evaluación para detectar degradaciones antes de que lleguen a producción.

Para recordar: Ninguna métrica automática captura toda la dimensionalidad de la calidad humana; la evaluación robusta de un sistema de IA requiere al menos tres métricas complementarias que cubran diferentes dimensiones más evaluación humana periódica como ground truth para validar que las métricas automáticas correlacionan con el juicio experto del dominio.

Módulo 5 – Capítulo 05 – Sección 05

Regression testing: detectar degradaciones entre versiones de modelos

Las regresiones en sistemas de IA son fundamentalmente diferentes a las regresiones en software tradicional: el sistema sigue respondiendo sin excepciones, sin errores de runtime, sin alertas de infraestructura; simplemente las respuestas son peores. Un sistema de soporte que responde con menos precisión, un asistente de redacción que produce texto menos fluido, un clasificador que acumula más errores en categorías específicas —estas degradaciones son invisibles para la monitorización técnica convencional y solo se detectan con evaluación sistemática de calidad. El regression testing en IA es el mecanismo que hace visibles estas degradaciones silenciosas antes de que los usuarios las reporten.

Las fuentes de regresión en sistemas de IA son más numerosas y frecuentemente externas al control del equipo que en software tradicional. Un cambio de prompt que mejora el caso de uso principal puede degradar el rendimiento en subgrupos de usuarios con patrones de consulta diferentes. Una actualización del proveedor al modelo —aunque el identificador permanezca idéntico— puede cambiar sutilmente el comportamiento en casos límite que el equipo no había testeado. Una actualización de la versión del framework de orquestación puede cambiar la forma en que se construyen los mensajes. Un cambio en el modelo de embeddings puede afectar la calidad del retriever en RAG. Cada uno de estos vectores de regresión requiere que la suite de evaluación se ejecute y compare contra el baseline antes de aceptar el cambio.

La estrategia de regression testing sigue un proceso de cuatro pasos. Primero, mantener un baseline registrado: el resultado de ejecutar la suite de evaluación sobre la versión en producción, con las métricas agregadas (media, P25, P75) y los scores por caso individual almacenados en `baseline_metrics.json` en el repositorio. Segundo, ejecutar la suite sobre la nueva versión: mismo dataset, mismos parámetros de evaluación, misma configuración de LLM evaluador. Tercero, comparar estadísticamente: no solo comparar medias sino distribuciones completas con tests estadísticos apropiados —Mann-Whitney U si el dataset tiene más de 30 muestras— para determinar si la diferencia es estadísticamente significativa o

dentro del ruido del proceso estocástico. Cuarto, bloquear o aprobar: si la diferencia es negativa y estadísticamente significativa, bloquear el merge del PR con evidencia específica de qué casos degradaron y por qué.

La herramienta más práctica para implementar regression testing en CI es una combinación de un script de evaluación centralizado — `evaluate.py --dataset tests/eval/dataset.json --model claude-3-5-sonnet-20241022 --output results.json` — con un script de comparación — `compare_metrics.py --baseline baseline_metrics.json --current results.json --threshold 0.03` — que falla con código de salida distinto de cero si alguna métrica degrada más del umbral configurado. DeepEval y LangSmith Datasets & Testing ofrecen esta comparación de forma integrada, con output compatible con JUnit XML para integración directa en GitHub Actions.

Aspectos técnicos del regression testing

- **Snapshot testing de prompts:** con `temperature=0` y seed fijo, almacenar el output del LLM para inputs de referencia; comparar en cada PR como señal de que algo cambió que requiere revisión; no como test de igualdad exacta sino como detector de cambio de comportamiento.
- **Comparación de distribuciones de métricas:** comparar media, P25 y P75 del score de calidad entre baseline y nueva versión; dos versiones con medias similares pero distribuciones diferentes pueden tener comportamientos muy distintos en casos límite.
- **Versionado de prompts como código:** archivos `.jinja2` o `.txt` en `prompts/` con versiones semánticas (`system_prompt_v1.2.0.jinja2`); el hash del archivo es la referencia al prompt exacto que produjo los resultados del baseline histórico.
- **CI job condicional:** ejecutar la suite de regression tests solo cuando los archivos de prompt, configuración del modelo o código del pipeline cambian; usar `paths` filters en GitHub Actions para activar el job selectivamente.
- **Alertas de degradación en producción:** además del regression testing en CI, monitorear métricas de calidad en tiempo real con Langfuse o LangSmith y disparar alertas cuando el score medio de un evaluador automático cae más del umbral definido en las últimas N horas.

Nota del Arquitecto: El momento más valioso de un sistema de regression testing no es cuando bloquea un PR problemático —que es su función visible— sino cuando un proveedor actualiza silenciosamente el comportamiento de un modelo y el

scheduled job de evaluación nocturno detecta la degradación antes de que ningún usuario la reporte. Este escenario —una degradación de origen externo, sin ningún cambio en el repositorio— es el que separa los sistemas que operan con evidencia de los que operan por sorpresa.

Las regresiones de calidad en sistemas de IA son silenciosas por naturaleza: el sistema continúa respondiendo sin errores de runtime mientras la calidad se degrada. Solo un sistema de evaluación automatizado con métricas de calidad y comparación estadística contra un baseline las detecta antes de que los usuarios las reporten.

Principio rector: Las regresiones de calidad en sistemas de IA son silenciosas: el sistema sigue respondiendo sin errores de runtime mientras la calidad se degrada; solo un sistema de evaluación automatizado con métricas de calidad y comparación estadística contra un baseline las detecta antes de que los usuarios las reporten.

Módulo 5 – Capítulo 05 – Sección 06

Cierre: pirámide de testing adaptada a sistemas de IA

La pirámide de testing clásica —muchos unit tests rápidos en la base, menos tests de integración en el nivel intermedio, pocos end-to-end en la cúspide— captura un principio fundamental de ingeniería: cuanto más bajo en la pirámide, más rápido, más barato y más preciso en la localización del fallo es el test; cuanto más alto, más realista pero más lento y más costoso. Para sistemas de IA, esta pirámide necesita una capa adicional que no tiene equivalente en el software tradicional: la evaluación de calidad, que existe entre los tests de integración y los end-to-end, y que es la capa específica de IA del modelo de calidad.

La pirámide adaptada para sistemas de IA tiene cuatro capas con características operacionales bien diferenciadas. En la base, los unit tests con mocks del LLM verifican la lógica de Python —constructores de prompts, parsers, validators, lógica de retry— en milisegundos y sin costo de API. Son más de cien tests que se ejecutan en cada commit y cada PR, con cobertura del 90% o más de las ramas del código Python. En la segunda capa, los tests de integración con LLMs reales validan los flujos completos sobre datasets curados de 20 a 100 casos, con LLMs económicos de test y `max_tokens` acotado; su costo por run es de centavos y su tiempo de ejecución de minutos; se ejecutan en PRs que modifican prompts, modelos o el pipeline de procesamiento. En la tercera capa, la evaluación de calidad con RAGAS, DeepEval o LLM-as-judge sobre 50 a 500 casos del dominio calcula métricas de faithfulness, relevancy y coherence; su costo por run es de uno a diez dólares y su tiempo de ejecución puede ser de decenas de minutos; se ejecuta en PRs relevantes y como scheduled job diario o nocturno. En la cúspide, la evaluación humana sobre 20 a 50 casos por anotadores con conocimiento del dominio valida que las métricas automáticas correlacionan con el juicio experto; su costo es de horas de trabajo humano y se ejecuta mensualmente o antes de releases mayores.

El triggering condicional basado en los archivos modificados es fundamental para que la pirámide completa sea operacionalmente viable. Si cada commit ejecutara la evaluación completa con 500 casos sobre un LLM costoso, el costo acumulado sería prohibitivo y los tiempos de feedback inaceptables. La lógica condicional es simple: `if "prompts/" in changed_files or "config/llm_config.yaml" in changed_files: run_evaluation_suite()`.

Esta condición ejecuta la suite costosa exactamente cuando es necesario —cuando algo que puede afectar la calidad del sistema cambió— y no en commits de documentación, tests o refactors de infraestructura que no pueden afectar la calidad de las respuestas del LLM.

El feedback loop cierra el ciclo: las respuestas evaluadas negativamente por usuarios en producción —thumbs down, reportes de feedback negativo, regeneraciones inmediatas de la misma respuesta— se incorporan al dataset de evaluación para que regresiones similares en el futuro sean detectadas automáticamente. Este crecimiento orgánico del dataset con los casos de fallo reales hace que el sistema de testing se vuelva progresivamente más específico para los problemas que el sistema real enfrenta, en lugar de quedarse estático en los casos anticipados durante el diseño inicial.

Capas de la pirámide de testing adaptada a IA

- **Base (unit tests, >100 tests):** mocks del LLM, pytest, cobertura del 90%+ de branches del código Python; costo \$0, tiempo <30 segundos; se ejecuta en cada commit y cada PR sin excepción.
- **Segunda capa (tests de integración, 20-100 casos):** LLM real económico, dataset curado, validación de propiedades y schema; costo \$0.01-\$1 por run; se ejecuta en PRs que modifican prompts, modelos o el pipeline de procesamiento.
- **Tercera capa (evaluación de calidad, 50-500 casos):** RAGAS, DeepEval o LLM-as-judge; métricas de faithfulness, relevancy, coherence; costo \$1-\$10 por run; se ejecuta en PRs relevantes y como scheduled job diario.
- **Cúspide (evaluación humana, 20-50 casos):** anotadores con conocimiento del dominio, rúbrica explícita con ejemplos ancla, inter-annotator agreement medido; costo en horas de trabajo; mensualmente o antes de releases mayores.
- **Feedback loop:** casos de fallo de producción (feedback negativo, regeneraciones) se añaden al dataset de la segunda y tercera capa; el dataset crece con los problemas reales del sistema, no solo con los anticipados.

La pirámide de testing de IA no es un artefacto que se construye al final del proyecto: se construye incrementalmente desde el primer sprint. El primer sprint tiene los unit tests con mocks. El segundo sprint, cuando el primer flujo completo funciona, añade los tests de integración. El tercero, cuando el sistema llega a los primeros usuarios reales, añade la evaluación de calidad. La evaluación humana se incorpora cuando el sistema tiene volumen suficiente para justificar el proceso. Esta secuencia incremental es la misma que aplica a

cualquier otro componente del sistema: mínimo viable primero, complejidad solo cuando la evidencia lo justifica.

"Testing leads to failure, and failure leads to understanding." — Burt Rutan. En AI Engineering, las respuestas incorrectas del sistema que se capturan como casos de test son el activo más valioso para construir un sistema que mejora continuamente; cada fallo documentado como caso de prueba es una garantía de que ese fallo específico no llegará a producción de nuevo.